Fenway Themed Tetris

Fenway-Inspired Tetris: A Modern Game Implementation

Tatiana Boothe, Camila Bautista

Boston University
EK327: Introduction to Software Engineering

Introduction

This project is a modern version of *Tetris*, built in Python and designed with a theme inspired by Fenway Park. The goal is to keep the original gameplay the same while improving the game's structure and appearance. The project focuses on building a clean and organized system, with a clear separation between the game logic and the visual interface. It also shows how core programming concepts in Python, like arrays, loops, and classes, can be used to create a fully working game.

Project Overview

The game lets players control falling blocks and place them on a grid. When a full row is formed, it disappears, and the player gets points. New pieces continue to fall, so the player has to keep placing them carefully to avoid filling up the board. Behind the scenes, the game uses a 2D array in Python to represent the board and classes to represent each piece. The program continuously updates the game by checking for movement, collisions, and completed rows. The frontend displays everything and handles user input, while the backend controls how the game works.

Goals and Motivation

The goal of this project was to build a fully functional game in Python that combines real-time interaction with structured design. We focused on creating a system that is organized, easy to understand, and able to handle continuous updates like movement, input, and scoring. This project also gave us the chance to apply class concepts more practically, especially in working with 2D arrays, object-oriented design, and game logic. Instead of solving isolated problems, we had to connect multiple components into one working system. While researching the creation of Tetris, we took a liking to the nostalgic pixelated images and fonts. In addition, we chose a Fenway-inspired theme to make the game visually distinct. This choice was also influenced by one of the team members being a Boston native, which made the theme more meaningful while still keeping the gameplay straightforward and familiar.

How the Project Works

Execution Flow

- A new piece is generated at the top of the board
- The player moves or rotates the piece using keyboard input
- The piece moves downward automatically over time
- If the piece reaches another block or the bottom, it is locked into place
- The board is checked for any fully completed rows
- Completed rows are removed, and the score is updated
- A new piece is generated and the process repeats

System Behavior

- A The board state is stored in a 2D array
- Each piece is represented as a matrix and updated based on user input
- The game loop continuously updates the state using time-based movement
- Collision checks ensure valid placement of pieces

- The system runs until the board fills to the top, ending the game

Algorithms Used

1 SPAWN PIECE

Randomly selects a shape, sets its color, and places it at the top center. Ends game if the position is invalid.

```
def spawnpiece(self):
    self.piece_shape = random.choice(SHAPES)
    self.piece_color_idx = next(
        val for row in self.piece_shape for val in row if val != 0
    )
    self.piece_x = BOARD_WIDTH // 2 - len(self.piece_shape[0]) // 2
    self.piece_y = 0
    if not self.valid(0, 0):
        self.game_over = True
```

2 COLLISION DETECTION (VALIDATION)

Checks if a move is valid by ensuring the piece stays within bounds and does not overlap existing blocks.

```
def valid(self, dx, dy, shape=None):
    shape = shape or self.piece_shape
    for r, row in enumerate(shape):
        for c, val in enumerate(row):
            if val:
                nx = self.piece_x + c + dx
                ny = self.piece_y + r + dy
                if nx < 0 or nx >= BOARD_WIDTH: return False
                if ny >= BOARD_HEIGHT: return False
                if ny >= 0 and self.board[ny][nx]: return False
    return True
```

3 MOVEMENT ALGORITHM

Updates the piece position only if the move is valid according to the collision detection algorithm.

```
def move(self, dx, dy):
    if not self.game_over and self.valid(dx, dy):
        self.piece_x += dx
        self.piece_y += dy
```

4 ROTATION ALGORITHM

Rotates the piece using a matrix transformation (transpose and reverse) and applies it only if the new position is valid.

```
def rotate(self):
    transfshape = [list(row) for row in zip(*self.piece_shape[::-1])]
    if self.valid(0, 0, transfshape):
        self.piece_shape = transfshape
```

5 HARD DROP & LOCK PIECE ALGORITHM

Drops the piece to the bottom as far as possible, then locks it permanently on the board.

```
def harddrop(self):
    while self.valid(0, 1):
        self.piece_y += 1
    self.lockpiece()
```

```
def lockpiece(self):
    for r, row in enumerate(self.piece_shape):
        for c, val in enumerate(row):
            if val:
                board_x = self.piece_x + c
                board_y = self.piece_y + r
                if board_y >= 0:
                    self.board[board_y][board_x] = self.piece_color_idx
self.clearlines()
self.spawnpiece()
```

6 ROW CLEARING & SCORE ALGORITHM

Removes completed rows, shifts the board down, and updates the score and round progress.

```
def clearlines(self):
    shiftedboard = [row for row in self.board if 0 in row]
    lines_cleared = BOARD_HEIGHT - len(shiftedboard)
    if lines_cleared > 0:
        self.score += lines_cleared * 100
        self.lines_in_round += lines_cleared
        if self.lines_in_round >= self.lines_per_round:
            self.nextround()
    for _ in range(lines_cleared):
        shiftedboard.insert(0, [0] * BOARD_WIDTH)
    self.board = shiftedboard
```

7 FALL TIMING (GRAVITY) ALGORITHM

Controls automatic downward movement of pieces using a time-based check.

```
def run(self):
    ...
    self.fall_time += delta_time
    if self.fall_time >= self.fall_speed:
        if not self.game_over:
            if self.valid(0, 1):
                self.piece_y += 1
            else:
                self.lockpiece()
        self.fall_time = 0
.....
```

8 NEXT ROUND / SPEED INCREASE ALGORITHM

Increases the round and makes pieces fall faster after a set number of cleared lines.

```
def nextround(self):
    self.round += 1
    self.lines_in_round = 0
    self.fall_speed = max(100, self.fall_speed - 50)
```

Key Code Components

The main code is organized around the TetrisEngine class, which controls the overall game.
It manages the board, pieces, input, rendering, scoring, and game state.
Below are the key components that work together to run the game.

1. Board Setup

Creates the game grid and initializes variables.

```
self.board = [[0 for _ in
      range(BOARD_WIDTH)]
     for _ in range(BOARD_HEIGHT)]
self.score = 0
self.game_over = False
self.game_started = False
self.paused = False
```

2. Piece Spawning

Randomly selects a shape and places it at the top.

```
self.piece_shape = random.choice(SHAPES)
self.piece_color_idx = next(
     val for row in self.piece_shape
     for val in row if val != 0
  )
self.piece_x = BOARD_WIDTH // 2 -
      len(self.piece_shape[0]) // 2
self.piece_y = 0
```

3. Movement & Rotation

Moves and rotates pieces if valid.

```
self.move(-1, 0)       # left
self.move(1, 0)        # right
self.move(0, 1)        # down
self.rotate()          # rotate
self.harddrop()        # instant drop
```

9. Game Loop

Main loop controlling updates and timing.

```
while running:
    dt = self.clock.tick(60)
    self.fall_time += dt
    if not self.game_over and not self.paused and self.game_started:
        if self.fall_time >= self.fall_speed:
            if self.valid(0, 1):
                self.piece_y += 1
            else:
                self.lockpiece()
            self.fall_time = 0
```

TetrisEngine Class

- Manages the board (self.board)
- Spawns and controls pieces
- Handles movement + rotation
- Checks valid positions (valid())
- Locks pieces and clears lines
- Handles input and game loop
- Draws game and UI screens

4. Collision Detection

Ensures moves are valid before applying them.

```
def valid(self, dx, dy, shape=None):
    if nx < 0 or nx >= BOARD_WIDTH:
        return False
    if ny >= BOARD_HEIGHT:
        return False
    if ny >= 0 and self.board[ny][nx]:
        return False
    return True
```

5. Locking Pieces

Places piece permanently on the board when it can't fall.

```
for r, row in enumerate(self.piece_shape):
    for c, val in enumerate(row):
        if val:
            board_x = self.piece_x + c
            board_y = self.piece_y + r
            if board_y >= 0:
                self.board[board_y][board_x] =
                  self.piece_color_idx
```

6. Line Clearing + Scoring

Removes full rows and updates score and rounds.

```
shiftedboard = [row for row in self.board
                if 0 in row]
lines_cleared = BOARD_HEIGHT - len(shiftedboard)
if lines_cleared > 0:
    self.score += lines_cleared * 100
    self.lines_in_round += lines_cleared
    if self.lines_in_round >= self.lines_per_round:
        self.nextround()
for _ in range(lines_cleared):
    shiftedboard.insert(0, [0] * BOARD_WIDTH)
self.board = shiftedboard
```

7. Drawing Functions

Handles all visuals (board + UI).

```
self.drawboard()
self.drawblock(x, y, color)
self.sidebar()
self.homescreen()
self.gameover()
self.pausedscreen()
```

8. Input Handling

Controls gameplay using keyboard.

```
if event.key == pygame.K_LEFT: self.move(-1, 0)
elif event.key == pygame.K_RIGHT: self.move(1, 0)
elif event.key == pygame.K_UP: self.rotate()
elif event.key == pygame.K_SPACE: self.harddrop()
elif event.key == pygame.K_p: self.paused = not self.paused
```

Design Trade Offs

We chose to use a 2D array to represent the board because it directly maps to the grid structure, making it easier to implement and debug. A more complex structure could allow additional features, but would add unnecessary complexity for this project.

We used a class-based design to separate the game logic from the interface. This improves organization and readability, but requires more planning compared to combining everything in one place.

For piece rotation, we used matrix transformations (transpose and reverse), which is efficient but less intuitive at first.

We initially built the interface using Tkinter, but later switched to Pygame. Tkinter was simpler to start with, but Pygame provided better support for real-time rendering and smoother gameplay, making it a

better fit for this project. Additionally, Pygame provided higher graphics and better display compared to Tkinter.

We also updated the home screen design from a simple color background to using an imported image. This made the interface more visually engaging, although it required additional setup for image loading and scaling.

We prioritized a simple interface over advanced graphics to ensure smooth performance and easier debugging.

We refined the visual design by narrowing the theme from a general Boston concept to a specific Fenway theme. This made the design more consistent and cohesive, even though it limited the range of visuals we could include.

Although this is not specific to the code itself, when listing the instructions, we had to improvise for the arrow keys. When initially writing the instructions in the code, we used the Arial font. This font allowed us to utilize ← ↑ → and ↓. However, we later downloaded a file from Google Fonts that fit our pixelated theme better. The font is called Jersey 10 and was created by Sarah Cadigan-Fried. The arrow symbols did not show up in this font, so we had to improvise and use other symbols.

Before - Arial Font	Finalized - Jersey10-Regular.ttf
← = move left	`<-  = move left`
→ = move left	`-> = move right`
↓ = Hard drop!	`v = Hard drop!`
↑ = rotate piece	`^ = rotate piece`

Team Contribution

Tatiana Boothe: Backend development (game logic, algorithms, board system)
Camila Bautista: Frontend and visual design (Fenway theme, user interface)
Both: Testing, planning, and documentation

There was significant overlap between roles, with both team members contributing across different parts of the project as needed, including debugging, feature development, and overall design decisions.

Project Scope and Complexity

This project involves building a complete system that runs in real time, handling game state, movement, collisions, and user input all at once. Since all parts must work together during execution, it adds more complexity than isolated tasks. Compared to other assignments, this project also allowed more freedom in design and implementation. We made our own decisions about structure, features, and the Fenway-themed visuals, making it more open-ended and fully developed.

Key Features and Impact

This project brings together core game mechanics like movement, collision handling, and row clearing into a single working system. Its impact comes from how these pieces are combined with a Fenway-inspired design to create something more engaging than a standard version. Small details, like including an image of David Price, add personality and connect the game to Boston, making it both functional and visually distinct.

Summary

This project implements a working version of *Tetris* in Python, combining structured game logic with a Fenway-inspired design. It demonstrates how multiple components—data structures, algorithms, and user interaction—can be integrated into a complete system.